

CuTe Layouts

This document describes `Layout`, CuTe’s core abstraction. Fundamentally, a `Layout` maps from coordinate space(s) to an index space.

`Layout`s present a common interface to multidimensional array access that abstracts away the details of how the array’s elements are organized in memory. This lets users write algorithms that access multidimensional arrays generically, so that layouts can change, without users’ code needing to change. For example, a row-major MxN layout and a column-major MxN layout can be treated identically in software.

CuTe also provides an “algebra of `Layout`s.” `Layout`s can be combined and manipulated to construct more complicated layouts and to tile layouts across other layouts. This can help users do things like partition layouts of data over layouts of threads.

Fundamental Types and Concepts

Integers

CuTe makes great use of dynamic (known only at run-time) and static (known at compile-time) integers.

- Dynamic integers (or “run-time integers”) are just ordinary integral types like `int` or `size_t` or `uint16_t`. Anything that is accepted by `std::is_integral<T>` is considered a dynamic integer in CuTe.
- Static integers (or “compile-time integers”) are instantiations of types like `std::integral_constant<Value>`. These types encode the value as a `static constexpr` member. They also support casting to their underlying dynamic types, so they can be used in expressions with dynamic integers. CuTe defines its own CUDA-compatible static integer types `cute::C<Value>` along with overloaded math operators so that math on static integers results in static integers. CuTe defines shortcut aliases `Int<1>`, `Int<2>`, `Int<3>` and `_1`, `_2`, `_3` as conveniences, which you should see often within examples.

CuTe attempts to handle static and dynamic integers identically. In the examples that follow, all dynamic integers could be replaced with static integers and vice versa. When we say “integer” in CuTe, we almost always mean a static OR dynamic integer.

CuTe provides a number of traits to work with integers.

- `cute::is_integral<T>`: Checks whether `T` is a static or dynamic integer type.
- `cute::is_std_integral<T>`: Checks whether `T` is a dynamic integer type. Equivalent to `std::is_integral<T>`.
- `cute::is_static<T>`: Checks whether `T` is an empty type (so instantiations cannot depend on any dynamic information). Equivalent to `std::is_empty`.
- `cute::is_constant<N, T>`: Checks that `T` is a static integer AND its value is equivalent to `N`.

See the [integral_constant](#) implementations for more information.

Tuple

A tuple is a finite ordered list of zero or more elements. The `cute::tuple` class behaves like `std::tuple`, but works on device and host. It imposes restrictions on its template arguments and strips down the implementation for performance and simplicity.

IntTuple

CuTe defines the `IntTuple` concept as either an integer, or a tuple of `IntTuples`. Note the recursive definition. In C++, we define [operations on IntTuple](#).

Examples of `IntTuple`s include:

- `int{2}`, the dynamic integer 2.
- `Int<3>{}`, the static integer 3.
- `make_tuple(int{2}, Int<3>{})`, the tuple of dynamic-2, and static-3.
- `make_tuple(uint16_t{42}, make_tuple(Int<1>{}, int32_t{3}), Int<17>{})`, the tuple of dynamic-42, tuple of static-1 and dynamic-3, and static-17.

CuTe reuses the `IntTuple` concept for many different things, including Shape, Stride, Step, and Coord (see [include/cute/layout.hpp](#)).

Operations defined on `IntTuple`s include the following.

- `rank(IntTuple)`: The number of elements in an `IntTuple`. A single integer has rank 1, and a tuple has rank `tuple_size`.
- `get<I>(IntTuple)`: The `I`th element of the `IntTuple`, with `I < rank`. For single integers, `get<0>` is just that integer.

- `depth(IntTuple)`: The number of hierarchical `IntTuple`s. A single integer has depth 0, a tuple of integers has depth 1, a tuple that contains a tuple of integers has depth 2, etc.
- `size(IntTuple)`: The product of all elements of the `IntTuple`.

We write `IntTuple`s with parentheses to denote the hierarchy. For example, `6`, `(2)`, `(4,3)`, and `(3,(6,2),8)` are all `IntTuple`s.

Shapes and Strides

Both `Shape` and `Stride` are `IntTuple` concepts.

Layout

A `Layout` is a tuple of (`Shape`, `Stride`). Semantically, it implements a mapping from any coordinate within the `Shape` to an index via the `Stride`.

Tensor

A `Layout` can be composed with data – e.g., a pointer or an array – to create a `Tensor`. The index generated by the `Layout` is used to subscript an iterator to retrieve the appropriate data. For details on `Tensor`, please refer to the [Tensor](#) section of the tutorial.

Layout Creation and Use

A `Layout` is a pair of `IntTuple`s: the `Shape` and the `Stride`. The first element defines the abstract *shape* of the `Layout`, and the second element defines the *strides*, which map from coordinates within the shape to the index space.

We define many operations on `Layout`s analogous to those defined on `IntTuple`.

- `rank(Layout)`: The number of modes in a `Layout`. Equivalent to the tuple size of the `Layout`'s shape.
- `get<I>(Layout)`: The `I`th sub-layout of the `Layout`, with `I < rank`.
- `depth(Layout)`: The depth of the `Layout`'s shape. A single integer has depth 0, a tuple of integers has depth 1, a tuple of tuples of integers has depth 2, etc.
- `shape(Layout)`: The shape of the `Layout`.
- `stride(Layout)`: The stride of the `Layout`.

- `size(Layout)`: The size of the `Layout` function's domain. Equivalent to `size(shape(Layout))`.
- `cosize(Layout)`: The size of the `Layout` function's codomain (not necessarily the range). Equivalent to `A(size(A) - 1) + 1`.

Hierarchical access functions

`IntTuple`s and `Layout`s can be arbitrarily nested. For convenience, we define versions of some of the above functions that take a sequence of integers, instead of just one integer. This makes it possible to access elements inside of nested `IntTuple` or `Layout` more easily. For example, we permit `get<I...>(x)`, where `I...` is a “C++ parameter pack” that denotes zero or more (integer) template arguments. These hierarchical access functions include the following.

- `get<I0,I1,...,IN>(x) := get<IN>(...(get<I1>(get<I0>(x)))...)`. Extract the `IN`th of the ... of the `I1`st of the `I0`th element of `x`.
- `rank<I...>(x) := rank(get<I...>(x))`. The rank of the `I...`th element of `x`.
- `depth<I...>(x) := depth(get<I...>(x))`. The depth of the `I...`th element of `x`.
- `shape<I...>(x) := shape(get<I...>(x))`. The shape of the `I...`th element of `x`.
- `size<I...>(x) := size(get<I...>(x))`. The size of the `I...`th element of `x`.

In the following examples, you'll see use of `size<0>` and `size<1>` to determine loops bounds for the 0th and 1st mode of a layout or tensor.

Constructing a Layout

A `Layout` can be constructed in many different ways. It can include any combination of compile-time (static) integers or run-time (dynamic) integers.

```

Layout s8 = make_layout(Int<8>{});
Layout d8 = make_layout(8);

Layout s2xs4 = make_layout(make_shape(Int<2>{}, Int<4>{}));
Layout s2xd4 = make_layout(make_shape(Int<2>{}, 4));

Layout s2xd4_a = make_layout(make_shape (Int< 2>{}, 4),
                              make_stride(Int<12>{}, Int<1>{}));
Layout s2xd4_col = make_layout(make_shape(Int<2>{}, 4),
                              LayoutLeft{});
Layout s2xd4_row = make_layout(make_shape(Int<2>{}, 4),
                              LayoutRight{});

Layout s2xh4 = make_layout(make_shape (2, make_shape (2, 2)),
                              make_stride(4, make_stride(2, 1)));
Layout s2xh4_col = make_layout(shape(s2xh4),
                              LayoutLeft{});

```

The `make_layout` function returns a `Layout`. It deduces the types of the function's arguments and returns a `Layout` with the appropriate template arguments. Similarly, the `make_shape` and `make_stride` functions return a `Shape` resp. `Stride`. CuTe often uses these `make_*` functions due to restrictions around constructor template argument deduction (CTAD) and to avoid having to repeat static or dynamic integer types.

When the `Stride` argument is omitted, it is generated from the provided `Shape` with `LayoutLeft` as default. The `LayoutLeft` tag constructs strides as an exclusive prefix product of the `Shape` from left to right, without regard to the `Shape`'s hierarchy. This can be considered a “generalized column-major stride generation”. The `LayoutRight` tag constructs strides as an exclusive prefix product of the `Shape` from right to left, without regard to the `Shape`'s hierarchy. For shapes of depth one, this can be considered a “row-major stride generation”, but for hierarchical shapes the resulting strides may be surprising. For example, the strides of `s2xh4` above could be generated with `LayoutRight`.

Calling `print` on each layout above results in the following

```

s8      :  _8:_1
d8      :  8:_1
s2xs4   :  (_2, _4):(_1, _2)
s2xd4   :  (_2, 4):(_1, _2)
s2xd4_a :  (_2, 4):(_12, _1)
s2xd4_col : (_2, 4):(_1, _2)
s2xd4_row : (_2, 4):(4, _1)
s2xh4   :  (2, (2, 2)):(4, (2, 1))
s2xh4_col : (2, (2, 2)):(_1, (2, 4))

```

The `Shape:Stride` notation is used quite often for `Layout`. The `_N` notation is shorthand for a static integer while other integers are dynamic integers. Observe that both `Shape` and

`Stride` may be composed of both static and dynamic integers.

Also note that the `Shape` and `Stride` are assumed to be *congruent*. That is, `Shape` and `Stride` have the same tuple profiles. For every integer in `Shape`, there is a corresponding integer in `Stride`. This can be asserted with

```
static_assert(congruent(my_shape, my_stride));
```

Using a Layout

The fundamental use of a `Layout` is to map between coordinate space(s) defined by the `Shape` and an index space defined by the `Stride`. For example, to print an arbitrary rank-2 layout in a 2-D table, we can write the function

```
template <class Shape, class Stride>
void print2D(Layout<Shape, Stride> const& layout)
{
    for (int m = 0; m < size<0>(layout); ++m) {
        for (int n = 0; n < size<1>(layout); ++n) {
            printf("%3d ", layout(m,n));
        }
        printf("\n");
    }
}
```

which produces the following output for the above examples.

```
> print2D(s2xs4)
 0   2   4   6
 1   3   5   7
> print2D(s2xd4_a)
 0   1   2   3
12  13  14  15
> print2D(s2xh4_col)
 0   2   4   6
 1   3   5   7
> print2D(s2xh4)
 0   2   1   3
 4   6   5   7
```

We can see static, dynamic, row-major, column-major, and hierarchical layouts printed here. The statement `layout(m,n)` provides the mapping of the logical 2-D coordinate (m,n) to the 1-D index.

Interestingly, the `s2xh4` example isn't row-major or column-major. Furthermore, it has three modes but is still interpreted as rank-2 and we're using a 2-D coordinate. Specifically, `s2xh4` has a 2-D multi-mode in the second mode, but we're still able to use a 1-D coordinate for that mode. More on this in the next section, but first we can generalize this another step. Let's use a 1-D coordinate and treat all of the modes of each layout as a single multi-mode. For instance, the following `print1D` function

```
template <class Shape, class Stride>
void print1D(Layout<Shape,Stride> const& layout)
{
    for (int i = 0; i < size(layout); ++i) {
        printf("%3d ", layout(i));
    }
}
```

produces the following output for the above examples.

```
> print1D(s2xs4)
 0   1   2   3   4   5   6   7
> print1D(s2xd4_a)
 0  12   1  13   2  14   3  15
> print1D(s2xh4_col)
 0   1   2   3   4   5   6   7
> print1D(s2xh4)
 0   4   2   6   1   5   3   7
```

Any multi-mode of a layout, including the entire layout itself, can accept a 1-D coordinate. More on this in the following sections.

CuTe provides more printing utilities for visualizing Layouts. The `print_layout` function produces a formatted 2-D table of the Layout's mapping.

```
> print_layout(s2xh4)
(2, (2,2)):(4, (2,1))
  0   1   2   3
+---+---+---+---+
0 | 0 | 2 | 1 | 3 |
+---+---+---+---+
1 | 4 | 6 | 5 | 7 |
+---+---+---+---+
```

The `print_latex` function generates LaTeX that can be compiled with `pdflatex` into a color-coded vector graphics image of the same 2-D table.

Vector Layouts

We define a vector as any `Layout` with `rank == 1`. For example, the layout `8:1` can be interpreted as an 8-element vector whose indices are contiguous.

```
Layout: 8:1
Coord : 0 1 2 3 4 5 6 7
Index : 0 1 2 3 4 5 6 7
```

Similarly, the layout `8:2` can be interpreted as an 8-element vector where the indices of the elements are strided by `2`.

```
Layout: 8:2
Coord : 0 1 2 3 4 5 6 7
Index : 0 2 4 6 8 10 12 14
```

By the above rank-1 definition, we also interpret layout `((4,2)):((2,1))` as a vector, since its shape is rank-1. The inner shape looks like a 4x2 row-major matrix, but the extra pair of parenthesis suggest we can interpret those two modes as a 1-D 8-element vector. The strides tell us that the first `4` elements are strided by `2` and then there are `2` of those first elements strided by `1`.

```
Layout: ((4,2)):((2,1))
Coord : 0 1 2 3 4 5 6 7
Index : 0 2 4 6 1 3 5 7
```

We can see the second set of `4` elements are duplicates of the first `4` with an extra stride of `1`.

Consider the layout `((4,2)):((1,4))`. Again, it's `4` elements strided by `1` and then `2` of those first elements strided by `4`.

```
Layout: ((4,2)):((1,4))
Coord : 0 1 2 3 4 5 6 7
Index : 0 1 2 3 4 5 6 7
```

As a function from integers to integers, it's identical to `8:1`. It's the identity function.

Matrix examples

Generalizing, we define a matrix as any `Layout` that is rank-2. For example,


```
Shape : (4,2)
Stride: (1,4)
 0  4
 1  5
 2  6
 3  7
```

is a 4x2 column-major layout with stride-1 down the columns and stride-4 across the rows, and

```
Shape : (4,2)
Stride: (2,1)
 0  1
 2  3
 4  5
 6  7
```

is a 4x2 row-major layout with stride-2 down the columns and stride-1 across the rows. Majorness is simply which mode has stride-1.

Just like the vector layouts, each of the modes of the matrix can also be split into *multi-modes*. This lets us express more layouts beyond just row-major and column-major. For example,

```
Shape: ((2,2),2)
Stride: ((4,1),2)
 0  2
 4  6
 1  3
 5  7
```

is also logically 4x2, with stride-2 across the rows but a multi-stride down the columns. The first `2` elements down the column have a stride of `4` and then there is a copy of those with stride-1. Since this layout is logically 4x2, like the column-major and row-major examples above, we can *still* use 2-D coordinates to index into it.

Layout Concepts

In this section, we'll introduce the coordinate sets that `Layout` accepts and how the coordinate mappings and index mappings are computed.

Layout compatibility

We say that layout A is *compatible* with layout B if the shape of A is compatible with the shape of B. Shape A is compatible with shape B if

- the size of A is equal to the size of B and
- all coordinates within A are valid coordinates within B.

For example:

- Shape 24 is NOT compatible with Shape 32.
- Shape 24 is compatible with Shape (4,6).
- Shape (4,6) is compatible with Shape ((2,2),6).
- Shape ((2,2),6) is compatible with Shape ((2,2),(3,2)).
- Shape 24 is compatible with Shape ((2,2),(3,2)).
- Shape 24 is compatible with Shape ((2,3),4).
- Shape ((2,3),4) is NOT compatible with Shape ((2,2),(3,2)).
- Shape ((2,2),(3,2)) is NOT compatible with Shape ((2,3),4).
- Shape 24 is compatible with Shape (24).
- Shape (24) is NOT compatible with Shape 24.
- Shape (24) is NOT compatible with Shape (4,6).

That is, *compatible* is a weak partial order on Shapes as it is reflexive, antisymmetric, and transitive.

Layouts Coordinates

With the notion of compatibility above, we emphasize that every `Layout` accepts multiple kinds of coordinates. Every `Layout` accepts coordinates for any `Shape` that is compatible with it. CuTe provides mappings between these sets of coordinates via a colexicographical order.

Thus, all Layouts provide two fundamental mappings:

- the map from an input coordinate to the corresponding natural coordinate via the `Shape`, and
- the map from a natural coordinate to the index via the `Stride`.

Coordinate Mapping

The map from an input coordinate to a natural coordinate is the application of a colexicographical order (reading right to left, instead of “lexicographical,” which reads left to right) within the `Shape`.

Take the shape `(3, (2, 3))`, for example. This shape has three coordinate sets: the 1-D coordinates, the 2-D coordinates, and the natural (h-D) coordinates.

1-D	2-D	Natural		1-D	2-D	Natural
0	(0, 0)	(0, (0, 0))		9	(0, 3)	(0, (1, 1))
1	(1, 0)	(1, (0, 0))		10	(1, 3)	(1, (1, 1))
2	(2, 0)	(2, (0, 0))		11	(2, 3)	(2, (1, 1))
3	(0, 1)	(0, (1, 0))		12	(0, 4)	(0, (0, 2))
4	(1, 1)	(1, (1, 0))		13	(1, 4)	(1, (0, 2))
5	(2, 1)	(2, (1, 0))		14	(2, 4)	(2, (0, 2))
6	(0, 2)	(0, (0, 1))		15	(0, 5)	(0, (1, 2))
7	(1, 2)	(1, (0, 1))		16	(1, 5)	(1, (1, 2))
8	(2, 2)	(2, (0, 1))		17	(2, 5)	(2, (1, 2))

Each coordinate into the shape `(3, (2, 3))` has two *equivalent* coordinates and all equivalent coordinates map to the same natural coordinate. To emphasize again, because all of the above coordinates are valid inputs, a Layout with Shape `(3, (2, 3))` can be used as if it is a 1-D array of 18 elements by using the 1-D coordinates, a 2-D matrix of 3x6 elements by using the 2-D coordinates, or a h-D tensor of 3x(2x3) elements by using the h-D (natural) coordinates.

The previous 1-D print demonstrates how CuTe identifies 1-D coordinates with a colexicographical ordering of 2-D coordinates. Iterating from `i = 0` to `size(layout)` and indexing into our layout with the single integer coordinate `i`, traverses the 2-D coordinates in this “generalized-column-major” order, even if the layout maps coordinates to indices in a row-major or more complex fashion.

The function `cute::idx2crd(idx, shape)` is responsible for the coordinate mapping. It will take any coordinate within the shape and compute the equivalent natural coordinate for that shape.

```

auto shape = Shape<_3, Shape<_2, _3>>{};
print(idx2crd( 16, shape));           // (1, (1, 2))
print(idx2crd(_16{}, shape));         // (-1, (-1, -2))
print(idx2crd(make_coord( 1, 5), shape)); // (1, (1, 2))
print(idx2crd(make_coord(_1{}, 5), shape)); // (-1, (1, 2))
print(idx2crd(make_coord( 1, make_coord(1, 2)), shape)); // (1, (1, 2))
print(idx2crd(make_coord(_1{}, make_coord(1, _2{})), shape)); // (-1, (1, -2))

```

Index Mapping

The map from a natural coordinate to an index is performed by taking the inner product of the natural coordinate with the `Layout`'s `Stride`.

Take the layout `(3, (2, 3)):(3, (12, 1))`, for example. Then a natural coordinate `(i, (j, k))` will result in the index `i*3 + j*12 + k*1`. The indices this layout computes are shown in the 2-D table below where `i` is used as the row coordinate and `(j, k)` is used as the column coordinate.

	0	1	2	3	4	5	<== 1-D col coord
	(0,0)	(1,0)	(0,1)	(1,1)	(0,2)	(1,2)	<== 2-D col coord (j,k)
0	0	12	1	13	2	14	
1	3	15	4	16	5	17	
2	6	18	7	19	8	20	

The function `cute::crd2idx(c, shape, stride)` is responsible for the index mapping. It will take any coordinate within the shape, compute the equivalent natural coordinate for that shape (if it is not already), and compute the inner product with the strides.

```

auto shape = Shape<_3, Shape<_2, _3>>{};
auto stride = Stride<_3, Stride<_12, _1>>{};
print(crd2idx( 16, shape, stride)); // 17
print(crd2idx(_16{}, shape, stride)); // _17
print(crd2idx(make_coord( 1, 5), shape, stride)); // 17
print(crd2idx(make_coord(_1{}, 5), shape, stride)); // 17
print(crd2idx(make_coord(_1{}, _5{}), shape, stride)); // _17
print(crd2idx(make_coord( 1, make_coord(1, 2)), shape, stride)); // 17
print(crd2idx(make_coord(_1{}, make_coord(1, _2{})), shape, stride)); // _17

```

Layout Manipulation

Sublayouts

Sublayouts can be retrieved with `layout<I...>`

```
Layout a    = Layout<Shape<_4, Shape<_3, _6>>>{}; // (4, (3, 6)):(1, (4, 12))
Layout a0   = layout<0>(a);                      // 4:1
Layout a1   = layout<1>(a);                      // (3, 6):(4, 12)
Layout a10  = layout<1, 0>(a);                   // 3:4
Layout a11  = layout<1, 1>(a);                   // 6:12
```

or `select<I...>`

```
Layout a    = Layout<Shape<_2, _3, _5, _7>>{}; // (2, 3, 5, 7):(1, 2, 6, 30)
Layout a13  = select<1, 3>(a);                // (3, 7):(2, 30)
Layout a01  = select<0, 1, 3>(a);              // (2, 3, 7):(1, 2, 30)
Layout a2   = select<2>(a);                    // (5):(6)
```

or `take<ModeBegin, ModeEnd>`

```
Layout a    = Layout<Shape<_2, _3, _5, _7>>{}; // (2, 3, 5, 7):(1, 2, 6, 30)
Layout a13  = take<1, 3>(a);                  // (3, 5):(2, 6)
Layout a14  = take<1, 4>(a);                  // (3, 5, 7):(2, 6, 30)
// take<1, 1> not allowed. Empty layouts not allowed.
```

Concatenation

A `Layout` can be provided to `make_layout` to wrap and concatenate

```
Layout a = Layout<_3, _1>{}; // 3:1
Layout b = Layout<_4, _3>{}; // 4:3
Layout row = make_layout(a, b); // (3, 4):(1, 3)
Layout col = make_layout(b, a); // (4, 3):(3, 1)
Layout q    = make_layout(row, col); // ((3, 4), (4, 3)):(1, 3), (3, 1)
Layout aa   = make_layout(a); // (3):(1)
Layout aaa  = make_layout(aa); // ((3)):(1)
Layout d    = make_layout(a, make_layout(a), a); // (3, (3), 3):(1, (1), 1)
```

or can be combined with `append`, `prepend`, or `replace`.

```

Layout a = Layout<_3, _1>{};           // 3:1
Layout b = Layout<_4, _3>{};           // 4:3
Layout ab = append(a, b);              // (3,4):(1,3)
Layout ba = prepend(a, b);             // (4,3):(3,1)
Layout c = append(ab, ab);              // (3,4,(3,4)):(1,3,(1,3))
Layout d = replace<2>(c, b);            // (3,4,4):(1,3,3)

```

Grouping and flattening

Layout modes can be grouped with `group<ModeBegin, ModeEnd>` and flattened with `flatten`.

```

Layout a = Layout<Shape<_2, _3, _5, _7>>{}; // (_2, _3, _5, _7):(_1, _2, _6, _30)
Layout b = group<0, 2>(a);                  // ((_2, _3), _5, _7):((_1, _2), _6, _30)
Layout c = group<1, 3>(b);                  // ((_2, _3), (_5, _7)):((_1, _2), (_6, _30))
Layout f = flatten(b);                      // (_2, _3, _5, _7):(_1, _2, _6, _30)
Layout e = flatten(c);                      // (_2, _3, _5, _7):(_1, _2, _6, _30)

```

Grouping, flattening, and reordering modes allows the reinterpretation of tensors in place as matrices, matrices as vectors, vectors as matrices, etc.

Slicing

`Layout`s can be sliced, but slicing is more appropriate to perform on `Tensor`s. See the [Tensor](#) section for slicing details.

Summary

- The `Shape` of a `Layout` defines its coordinate space(s).
 - Every `Layout` has a 1-D coordinate space. This can be used to iterate over the coordinate spaces in a colexicographical order.
 - Every `Layout` has a R-D coordinate space, where R is the rank of the layout. The colexicographical enumeration of the R-D coordinates correspond to the 1-D coordinates above.
 - Every `Layout` has an h-D (natural) coordinate space where h is “hierarchical.” These are ordered colexicographically and the enumeration of that order corresponds to the 1-D coordinates above. A natural coordinate is *congruent* to the `Shape` so that each element of the coordinate has a corresponding element of the `Shape`.
- The `Stride` of a `Layout` maps coordinates to indices.

- The inner product of the elements of the natural coordinate with the elements of the `Stride` produces the resulting index.

For each `Layout` there exists an integral `Shape` that is that compatible with that `Layout`. Namely, that integral shape is `size(layout)`. We can then observe that

Layouts are functions from integers to integers.

If you're familiar with the C++23 feature `mdspan`, this is an important difference between `mdspan` layout mappings and CuTe `Layout`s. In CuTe, `Layout` is a first class citizen, is natively hierarchical to naturally represent functions beyond row-major and column-major, and can similarly be indexed with a hierarchy of coordinates. (`mdspan` layout mappings can represent hierarchical functions as well, but this requires defining a custom layout.) Input coordinates for an `mdspan` must have the same shape as the `mdspan`; a multidimensional `mdspan` does not accept 1-D coordinates.

Copyright

Copyright (c) 2017 - 2025 NVIDIA CORPORATION & AFFILIATES. All rights reserved. SPDX-



Copyright © 2025, NVIDIA Corporation.

Last updated on Jun 10, 2025.